Abstract

Heart disease remains one of the top causes of mortality worldwide, making early and precise prediction vital for successful therapeutic decision-making. This study intends to construct and assess predictive models for heart disease classification using both classic machine learning approaches and Artificial Neural Networks (ANN). The Cleveland Heart Disease dataset was utilized, encompassing a range of clinical and demographic variables essential to diagnosis.

A structured machine learning pipeline was created, comprising data preprocessing, feature encoding, and model training. Multiple classification models, including Logistic Regression, Support Vector Machine, Random Forest, Decision Tree, AdaBoost, and XGBoost, were built and enhanced utilizing hyperparameter tuning approaches. Model performance was evaluated using accuracy, precision, recall, F1-score, and ROC-AUC metrics, coupled with visualization tools such as confusion matrices and ROC curves.

An ANN model was later constructed for binary classification, combining hidden layers, dropout regularization, and sigmoid activation in the output layer. A validation-based training technique with early stopping was utilized to improve generalization, and the model was further refined through a two-stage hyperparameter tweaking process.

The results reveal that Logistic Regression had the highest overall accuracy (about 0.8947), while the ANN model demonstrated higher classification capabilities in terms of ROC-AUC (nearly 0.95), highlighting its ability to distinguish between classes across varying thresholds. Feature importance analysis employing permutation important identified critical predictors such as the number of major vessels, maximal heart rate, and electrocardiographic data.

The findings of this study reveal that while standard machine learning models are highly effective for structured datasets, ANN models offer superior generalization and classification performance. These results underscore the necessity of selecting models based on dataset properties, interpretability criteria, and application-specific demands.

Table of Contents

Course Submission Cover Sheet....1
1. Introduction....8
1.1 Project Background....8
1.2 Group Work Methodology....9
1.2.1 Data Preprocessing....9
1.2.2 Exploratory Data Analysis (EDA)....9
1.2.3 Baseline Machine Learning Models....10
1.2.4 Model Evaluation and ROC Curves....11
1.2.5 Hyperparameter Tuning....12
1.2.6 Model Comparison and Key Findings....12
1.2.7 Visualization and Interpretation....13
2. Introduction to Artificial Neural Network....14
2.1 Overview of Artificial Neural Networks....14
2.2 Structure and Architecture of ANN....15
2.3 Working Mechanism of ANN....15
2.4 Activation Functions in ANN....16
2.5 ANN in Binary Classification....16
2.6 Advantages of Artificial Neural Networks....17
2.7 Limitations of Artificial Neural Networks....17
2.8 Relevance of ANN to This Study....18
3. Data Preparation for Artificial Neural Networks....18
3.1 ANN Environmental Setup....18
3.2 Creation of Validation Dataset....19
4. Baseline ANN Model....20
4.1 Model Architecture....20
4.2 Model Compilation....21
4.3 Model Training....22

4.4 Baseline ANN Evaluation....23
4.5 Confusion Matrix Analysis....23
5. Hyperparametric Tuning for ANN....24
5.1 Tuning Strategy....24
5.2 ANN Tuning Round 1 - Architecture Selection....24
5.3 ANN Tuning Round 2 - Fine Tuning....26
5.4 Selecting the Final Best Tuned ANN Model....27
5.5 Discussion of Tuning Results....28
6. Final Selected ANN Model Evaluation....28
6.1 Performance Matrix....28
6.2 Confusion Matrix Analysis....29
6.3 Training and Validation Curves....30
6.4 ROC Curve Analysis....32
6.5 Comparison across Baseline vs Tuned ANN....33
7. Feature Importance and Interpretability....34
7.1 Importance of Model Interpretability in ANN....34
7.2 Computation of ANN Feature Importance using Permutation Method....34
7.3 Interpretation of ANN Feature Importance Results....36
7.4 Feature Importance in Tree-Based Models....36
7.5 Aggregation and Selection of Top Features....37
7.6 Comparison of Feature Importance Across Models....38
7.7 Justification for Using Permutation Importance....39
7.8 Key Insights from Feature Performance....39
8. Overall Model Comparison across All Tuned ML Models and Tuned ANN Model....40
8.1 Consolidated Performance Evaluation....40
8.2 Metric-wise Comparative Analysis....41
8.3 ROC Curve Comparison....42
8.4 Model Ranking and Best Model Selection....42

8.5 Key Insights from Model Comparison 43
8.6 Trade-offs Between Models 44
8.7 Overall Conclusion from Comparison 45
9. Overall Results, Insights, Challenges and Conclusion 46
9.1 Summary of Results 46
9.2 Overall Key Insights 46
9.3 Challenges and Limitations 47
9.4 Final Conclusion 48
References 49

Table of Figures

Figure 1: ANN libraries and dependencies used for model development. 19
Figure 2: Final dataset split showing training, validation, and test sizes. 20
Figure 3: Baseline ANN architecture defining layers, neurons, and activation function.. 21
Figure 4: Summary of the baseline ANN model showing layer outputs and parameters. 21
Figure 5:Training log showing accuracy and loss progression during baseline ANN training. 22
Figure 6:Performance metrics of the baseline ANN model on test data. 23
Figure 7:Confusion matrix of the baseline ANN model. 24
Figure 8:Hyperparameter search space defined for ANN tuning across different architectures. 26
Figure 9:Results of ANN hyperparameter tuning (Round 1) showing validation performance across configurations. 26
Figure 10:Ranked ANN tuning results with the best-performing model identified based on average evaluation metrics. 26
Figure 11:Refined hyperparameter grid for ANN tuning (Round 2) focusing on optimal parameter ranges. 27
Figure 12:Results of ANN hyperparameter tuning (Round 2) showing improved validation performance across configurations. 27
Figure 13:Final ranked ANN tuning results highlighting performance across evaluation metrics. 28
Figure 14:Final performance metrics of the tuned ANN model.. 29
Figure 15:Confusion matrix of the final tuned ANN model. 30
Figure 16:Training and validation loss curves of the tuned ANN model. 31
Figure 17:Training and validation accuracy curves of the tunedl ANN model. 32
Figure 18:ROC curve of the final ANN model.. 33
Figure 19:Comparison of baseline and tuned ANN performance across evaluation metrics. 34
Figure 20:Permutation importance scores for all features in the ANN model. 35
Figure 21:Top 12 important features based on ANN permutation importance. 36
Figure 22:Feature importance comparison across tree-based models and ANN (sample features).37
Figure 23:Top features selected based on aggregated importance across models. 38
Figure 24:Comparison of feature importance across tree-based models and ANN. 39
Figure 25:Performance comparison of all tuned ML models and the final ANN across evaluation metrics. 41
Figure 26:Comparison of model performance across accuracy, precision, recall, F1-score, and ROC-AUC. 41
Figure 27:ROC curve comparison of all tuned ML models and the ANN model. 42
Figure 28:Final ranking of models based on average performance across all evaluation metrics. . 43

1. Introduction

1.1 Project Background

Heart disease remains one of the leading causes of mortality worldwide, rendering early detection and precise diagnosis a significant problem in contemporary healthcare. Advancements in data-driven technologies have positioned machine learning and artificial intelligence as formidable instruments for enhancing medical decision-making by uncovering patterns in clinical data that traditional methods may overlook.

The project aims to forecast the occurrence of heart disease utilizing the Cleveland Heart Disease dataset, which encompasses many clinical and demographic attributes, including age, cholesterol levels, resting blood pressure, and electrocardiographic findings. The target variable is binary, signifying the presence (1) or absence (0) of cardiac disease in a patient. The aim of this study is to create predictive models that can precisely categorize patients according to these characteristics.

The project comprises two primary components: group work and individual tasks. The group component encompasses the execution and assessment of various multiple machine learning models, such as Logistic Regression, Random Forest, Support Vector Machine, Decision Tree, AdaBoost, and XGBoost. The models are trained through a systematic pipeline encompassing data preprocessing, exploratory data analysis, feature engineering, and performance evaluation.

A thorough evaluation framework is implemented utilizing essential performance indicators including accuracy, precision, recall, F1-score, and ROC-AUC. In addition, confusion matrices and ROC curves are employed to provide deeper insights into model performance. To further boost model effectiveness, hyperparameter tuning approaches such as GridSearchCV and RandomizedSearchCV are utilized to optimize each model.

The group work also stresses visualization and interpretability, combining feature importance analysis and comparison performance charts to better understand the behavior of different algorithms. These stages allow a thorough assessment of models and assist determine the most suited approach for heart disease prediction.

Building upon the group effort, the individual component of this project focuses on the building of an Artificial Neural Network (ANN) model. This enables for a comparison between

Traditional machine learning techniques and deep learning approaches, providing insights into their relative strengths, limits, and application in healthcare prediction problems.

Overall, this study aims to evaluate the effectiveness of different predictive modeling techniques and determine the most suitable approach for accurately diagnosing heart disease, while also highlighting the importance of model evaluation, optimization, and interpretability in realworld applications.

1.2 Group Work Methodology

1.2.1 Data Preprocessing

Data pre-processing was a vital step to ensuring the dataset was clean, consistent, and acceptable for machine learning models. Initially, the dataset was reviewed for missing values and duplicates. It was noticed that there were no missing values and no duplicate records, assuring data integrity.

Outlier detection was accomplished using the Interquartile Range (IQR) approach on important numerical parameters such as resting blood pressure (trestbps), cholesterol (chol), maximal heart rate (thalach), and ST depression (oldpeak). A total of 19 outliers were detected and deleted, lowering the sample size from 303 to 284 records, hence boosting model dependability.

Categorical factors such as sex, chest pain kind, and thalassemia were encoded using onehot encoding, which raised the number of features from 14 to 29. This allows machine learning models to analyze categorical data successfully.

Finally, numerical features were normalized using StandardScaler, ensuring that all features were on a comparable scale and preventing bias in models sensitive to feature magnitude.

1.2.2 Exploratory Data Analysis (EDA)

Exploratory data analysis was undertaken to better understand the dataset and uncover patterns, distributions, and correlations between variables.

Univariate analysis found that the dataset was generally balanced, with around 54% of patients having no heart disease and 46% having heart disease. Various visualizations such as bar charts, pie charts, and histograms were utilized to assess individual feature distributions.

Bivariate analysis was used to explore the relationship between independent factors and the target variable. For example, chest pain type and exercise-induced angina revealed high relationships with heart disease presence, indicating their usefulness as predictive features.

A correlation heatmap was also developed to explore correlations across variables and uncover potential multicollinearity. This assisted in identifying which features contributed most significantly to the prediction task.

1.2.3 Baseline Machine Learning Models

To generate a robust benchmark for performance comparison, six standard machine learning models were developed using the pre-processed dataset. These models were picked to demonstrate several learning methodologies, including linear models, tree-based methods, ensemble techniques, and kernel-based algorithms.

The models utilized were:

- Logistic Regression (linear model)
- Random Forest (ensemble, bagging-based)
- Support Vector Machine (kernel-based)
- Decision Tree (tree-based model)
- AdaBoost (boosting ensemble approach)
- XGBoost (advanced gradient boosting method)

Each model was trained using the same training dataset to ensure fairness in comparison. The dataset was split into training and testing sets using an 80:20 ratio, with stratification used to maintain the distribution of the target variable.

To analyse model performance completely, five main metrics were used:

- Accuracy - assesses overall correctness
- Precision - assesses correctness of positive forecasts
- Recall - evaluates ability to discern true positives
- F1-Score - harmonic mean of precision and recall ROC-AUC - assesses classification performance across thresholds

Among all baseline models, Logistic Regression achieved the highest performance, with an accuracy of roughly 87.72% and a ROC-AUC score of 0.9338 . This shows that the dataset has rather strong linear separability.

Support Vector Machine also performed well, displaying balanced precision and recall. Ensemble models such as Random Forest and AdaBoost exhibited stable performance but did not outperform Logistic Regression.

On the other side, the Decision Tree model demonstrated the lowest performance, possibly due to its tendency to overfit the training data when not appropriately tuned.

Overall, the basic models offered a robust foundation for additional optimization through hyperparameter tuning.

1.2.4 Model Evaluation and ROC Curves

A rigorous evaluation process was devised to guarantee that model performance was reviewed from different angles rather than relying just on accuracy.

In addition to the normal assessment measures, ROC-AUC curves were integrated for all models. The inclusion of ROC curves allows for a greater understanding of each model's classification capability across varied threshold settings. The ROC curve shows the True Positive Rate (Recall) against the False Positive Rate, while the Area Under the Curve (AUC) offers a single-value assessment of model performance.

A higher ROC-AUC value shows greater discriminating between classes. In this project, Logistic Regression achieved one of the best ROC-AUC values (~0.93), suggesting significant predictive capability.

Furthermore, confusion matrices were constructed for all models, offering a visual depiction of classification results:

- True Positives (properly anticipated illness cases)
- True Negatives (properly anticipated non-disease cases)
- False Positives
- False Negatives

This helped identify whether models were biased toward a particular class and allowed for improved interpretation of classification errors.

By combining numerical indicators, ROC curves, and confusion matrices, the evaluation process ensured a holistic and reliable comparison of all models.

1.2.5 Hyperparameter Tuning

After evaluating the baseline models, hyperparameter tuning was performed to maximize model performance and reduce potential concerns such as overfitting or underfitting.

Two tuning approaches were used:

- GridSearchCV - thorough search across predetermined parameter combinations
- RandomizedSearchCV - random sampling of parameter space (used for XGBoost owing to complexity)

Hyperparameter tuning was accomplished using cross-validation (5-fold) on the training dataset to achieve robust and unbiased optimization.

Key parameters tuned included:

- Logistic Regression: regularization strength (C), penalty type
- Random Forest: number of trees, maximum depth, minimum samples split
- SVM: kernel type, gamma, regularization parameter
- Decision Tree: depth, splitting criteria
- AdaBoost: number of estimators, learning rate
- XGBoost: learning rate, depth, regularization terms, subsampling

After tuning, most models showed improvement in performance.

The best-performing model remained Logistic Regression, with improved metrics:

- Accuracy ≈ 89.47%
- Precision ≈ 0.913 ROC-AUC ≈ 0.9325

Random Forest and SVM significantly improved following adjustment, particularly in precision and recall balance.

However, certain models such as AdaBoost and XGBoost exhibited minor losses in accuracy, demonstrating that tuning does not always guarantee improvement and must be properly verified.

Overall, hyperparameter tuning boosted model performance and ensured optimal setups for each method.

1.2.6 Model Comparison and Key Findings

Following hyperparameter tuning, a full comparison of all models was undertaken using all five evaluation measures. Instead, than depending on a single parameter, models were graded based on their overall performance across accuracy, precision, recall, F1-score, and ROC-AUC.

The results revealed that:

- Logistic Regression consistently ranked first across most measures, suggesting great generalization and stability
- Support Vector Machine performed as the second-best model, displaying balanced and consistent performance
- Random Forest displayed good robustness but significantly worse accuracy
- Ensemble models (AdaBoost and XGBoost) offered steady results but did not beat simpler models
- Decision Tree revealed symptoms of overfitting, as indicated by a higher training accuracy relative to testing accuracy

The ultimate ranking of models was:

- Logistic Regression
- Support Vector Machine
- Random Forest
- AdaBoost
- XGBoost
- Decision Tree

A significant finding from this comparison is that simpler models (like Logistic Regression) can beat sophisticated models when the dataset is well-structured and not very nonlinear.

Additionally, cross-validation findings indicated that the top-performing models had constant performance across different data splits, showing reliability.

1.2.7 Visualization and Interpretation

To enhance the interpretability of the models and support the analytical conclusions, numerous visualization techniques were applied.

1. Feature Importance

For tree-based models such as Random Forest, Decision Tree, AdaBoost, and XGBoost, feature significance scores were generated and shown.

These plots highlighted the most relevant elements contributing to heart disease prediction, helping to pinpoint critical clinical markers such as cholesterol levels, chest discomfort kind, and maximal heart rate.

2. ROC Curve Comparison

ROC curves were presented for all models on a single graph, allowing easy visual comparison of classification performance.

Models with curves closer to the top-left corner performed better. Logistic Regression and SVM provided superior ROC curves with higher AUC values. This effectively illustrated the relative merits of each model.

3. Performance Comparison Charts

Bar charts were utilized to compare model performance across all evaluation metrics. This allowed for:

- Easy identification of the best-performing model
- Visual comparison of strengths and drawbacks of each algorithm

4. Model Ranking Visualization

A ranking system was constructed by averaging the ranks of models across all metrics. This provided a more balanced review rather than relying on a single metric.

Overall Interpretation

The visualizations performed a key role in turning numerical results into relevant insights. They helped:

- Identify the most effective models
- Understand feature importance
- Validate model consistency
- Communicate findings clearly

These visual tools dramatically improved the interpretability and presentation of the results, making it easier to draw meaningful conclusions from the analysis.

This section first summarizes the group-based machine learning pipeline, followed by the individual ANN implementation and comparison.

2. Introduction to Artificial Neural Network

2.1 Overview of Artificial Neural Networks

Artificial Neural Networks (ANNs) are computational models inspired by the structure and functioning of the human brain, designed to learn complex patterns from data. They are supposed

to imitate the way organic neurons process information, enabling robots to learn complicated patterns from data. ANNs are particularly good at handling problems where interactions between variables are nonlinear or very complex, making them suitable for applications such as medical diagnosis, image recognition, and predictive analytics (Haykin, 2009).

Unlike standard machine learning algorithms, which typically rely on preexisting assumptions about data structure, ANNs are capable of spontaneously learning feature representations straight from the data. This feature allows them to capture hidden associations that may not be easily recognized by conventional statistical methods (Goodfellow, et al., 2016). In this study, ANN is applied for binary classification to predict the presence of heart disease and is compared with traditional machine learning models.

2.2 Structure and Architecture of ANN

An Artificial Neural Network is constructed of numerous layers of interconnected nodes, generally referred to as neurons. These layers are often separated into three primary components:

- Input Layer: The input layer is responsible for receiving raw input data. Each neuron in this layer reflects a feature from the dataset, such as age, cholesterol level, or blood pressure in the context of heart disease prediction.
- Hidden Layer(s): Hidden layers are the center of the neural network, where most of the computations take place. These layers change incoming data through weighted connections and activation functions, letting the model to learn complicated patterns and relationships. The presence of numerous hidden layers allows the network to generate hierarchical representations of data, a concept generally referred to as deep learning (LeCun, et al., 2015).
- Output Layer: The output layer produces the final forecast. In binary classification applications, such as detecting the existence of heart disease, a single neuron is often utilized with a Sigmoid activation function to output a probability value between 0 and 1 (Bishop, 2006).

2.3 Working Mechanism of ANN

The operation of an ANN requires a sequence of computational stages that enable it to learn from data.

- Forward Propagation: In forward propagation, input data moves through the network layer by layer. Each neuron computes a weighted sum of its inputs, adds a bias term, and applies an activation function to produce an output.
- Loss Calculation: The anticipated output is compared with the actual target value using a loss function, such as binary cross-entropy in classification tasks. This loss shows the mistake in forecast.
- Backpropagation: Backpropagation is the process of propagating the error backward through the network to update the weights. This is performed by calculating gradients of the loss function with respect to each weight.
- Optimization: An optimization procedure, such as gradient descent, is used to update the weights in a way that minimizes the loss function. This approach is continued iteratively until the model converges to an optimal solution (Goodfellow, et al., 2016).

2.4 Activation Functions in ANN

Activation functions have a critical role in injecting nonlinearity into the network, helping it to learn complicated linkages.

- ReLU (Rectified Linear Unit) is often employed in hidden layers due to its computing efficiency and ability to prevent vanishing gradient concerns.
- Sigmoid function is utilized in the output layer for binary classification, as it turns outputs into probabilities between 0 and 1.
- Tanh function is sometimes employed in hidden layers, yielding outputs between -1 and 1.

The choice of activation function strongly influences model performance and learning efficiency (Chollet, 2017).

2.5 ANN in Binary Classification

In this study, ANN is applied to a binary classification issue, where the goal is to predict if a patient has heart disease or not.

The output layer uses a Sigmoid activation function defined as:

$$\sigma(x) = \frac{1}{1 + e}$$

This function converts the output to a probability value between 0 and 1. A threshold, commonly 0.5, is used to classify the output into two categories:

- 0 represents no heart disease
- 1 signifies the presence of cardiac disease

This probabilistic approach provides for more flexible decision-making and evaluation using measures such as ROC-AUC (Bishop, 2006).

2.6 Advantages of Artificial Neural Networks

Artificial Neural Networks have various characteristics that make them useful tools for predictive prediction.

- They are capable of modelling nonlinear relationships and capturing intricate interactions between variables that traditional models may fail to discover.
- They can automatically learn feature representations straight from data without requiring manual feature engineering.
- When appropriately configured, they can achieve great predictive performance, especially in big and complicated datasets (LeCun, et al., 2015).

2.7 Limitations of Artificial Neural Networks

Despite their merits, ANNs also have some drawbacks.

- Training neural networks can be computationally expensive and time-consuming.
- They are generally termed black-box models, making it difficult to explain how predictions are created.
- Their performance is highly sensitive to hyperparameters such as learning rate, number of layers, and number of neurons.

- Without sufficient regularization, ANNs are prone to overfitting and may fail to generalize well to unknown data (Rudin, 2019).

2.8 Relevance of ANN to This Study

In the domain of heart disease prediction, ANNs provide an opportunity to test whether deep learning techniques may outperform traditional machine learning models. While simpler models such as Logistic Regression may perform well on structured datasets, ANNs have the potential to identify deeper patterns and interactions between clinical characteristics.

By constructing an ANN model and comparing its performance with traditional models, this study intends to examine the trade-offs between interpretability, complexity, and predictive accuracy in healthcare applications.

3. Data Preparation for Artificial Neural Networks

3.1 ANN Environmental Setup

Before the implementation of the Artificial Neural Network (ANN), the necessary libraries and frameworks were imported to facilitate model building, training, assessment, and visualization. The ANN model was developed with the TensorFlow and Keras packages, which offer a high-level interface for creating deep learning architectures.

Additional libraries, including Scikit-learn, were employed for dataset partitioning and performance assessment, whereas Matplotlib and Seaborn were utilized for visualization purposes. Moreover, sophisticated training mechanisms like EarlyStopping and ReduceLROnPlateau were integrated to enhance model efficacy and mitigate overfitting.

The implementation utilized the following components:

- Construction of neural networks with Sequential models and Dense layers
- Regularization methods including Dropout and L2 regularization
- Optimization with the Adam optimizer
- Evaluation measures including accuracy, precision, recall, F1-score, and ROC-AUC
- Training callbacks for performance monitoring and learning rate adjustment

These tools provided a complete framework for creating and optimizing the ANN model in a systematic and efficient manner.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization, Input
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.regularizers import l2
from tensorflow.keras.optimizers import Adam

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, classification_report, confusion_matrix,
    ConfusionMatrixDisplay, RocCurveDisplay, roc_curve, auc
)

from sklearn.inspection import permutation_importance

import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
import seaborn as sns

tf.random.set_seed(42)
np.random.seed(42)
```

Figure 1: ANN libraries and dependencies used for model development.

3.2 Creation of Validation Dataset

Before training the ANN model, the dataset was further prepared by producing a validation set from the training data. This stage is critical in deep learning models to check performance during training and to guarantee that the model generalizes effectively to unseen data.

Instead of immediately training the model on the whole training dataset, the data was separated into two subsets: a smaller training set and a validation set. This was done utilizing an 80:20 split, where 80% of the data was utilized for training and 20% was used for validation. Stratified sampling was performed to preserve the distribution of the target variable across both categories, ensuring that the proportion of patients with and without cardiac disease remained stable.

As demonstrated in the implementation, the resulting dataset sizes were:

- Training set: 181 samples with 28 characteristics
- Validation set: 46 samples with 28 characteristics

- Test set: 57 samples with 28 features

The introduction of a validation set plays a significant function in the ANN training process. It allows the model to be tested on unseen data during training, enabling the application of approaches such as early halting and hyperparameter tuning. This helps prevent overfitting and ensures that the final model performs well on new data.

Additionally, all features used for ANN training were already standardized during the preprocessing stage. Feature scaling is particularly crucial for neural networks, since it ensures steady and efficient learning by preventing excessive changes in input values.

Overall, this data preparation method ensures a solid training procedure and facilitates fair comparison with traditional machine learning models. This technique ensures that the test dataset stays fully unobserved during model construction, giving an unbiased evaluation of the final ANN model.

```
Training shape   : (181, 28)
Validation shape : (46, 28)
Test shape       : (57, 28)
```

Figure 2: Final dataset split showing training, validation, and test sizes.

4. Baseline ANN Model

4.1 Model Architecture

The baseline Artificial Neural Network (ANN) model was built to establish a reference point for performance comparison before performing hyperparameter tuning. A somewhat simple design was used to maintain fairness when comparing the ANN model with traditional machine learning models.

The model was developed utilizing a Sequential design, where layers are added in a linear fashion. As stated in the implementation, the network consists of:

- An input layer corresponding to 28 characteristics
- A first hidden layer with 32 neurons employing the ReLU activation function
- A dropout layer with a rate of 0.20 to reduce overfitting
- A second hidden layer with 16 neurons employing the ReLU activation function
- A second dropout layer with a rate of 0.10
- An output layer with a single neuron employing the Sigmoid activation function

This architecture was developed to reconcile model complexity and generalization. The introduction of dropout layers helps prevent overfitting by randomly deactivating neurons during training, while the Sigmoid activation function in the output layer permits probability-based binary classification.

```
baseline_ann_model = Sequential([
    Input(shape=(X_train_ann.shape[1],)),

    # Simple baseline network for fair comparison before ANN tuning
    Dense(32, activation='relu'),
    Dropout(0.20),

    Dense(16, activation='relu'),
    Dropout(0.10),

    Dense(1, activation='sigmoid')
])
```

Figure 3: Baseline ANN architecture defining layers, neurons, and activation function..

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 32)	928
dropout (Dropout)	(None, 32)	0
dense_1 (Dense)	(None, 16)	528
dropout_1 (Dropout)	(None, 16)	0
dense_2 (Dense)	(None, 1)	17

Total params: 1,473 (5.75 KB)
Trainable params: 1,473 (5.75 KB)
Non-trainable params: 0 (0.00 B)

Figure 4: Summary of the baseline ANN model showing layer outputs and parameters.

4.2 Model Compilation

After describing the architecture, the model was compiled by specifying the optimizer, loss function, and evaluation metrics.

The Adam optimizer was employed with a learning rate of 0.001, as it provides efficient and adaptive learning for neural networks. The loss function selected was binary cross-entropy, which is ideal for binary classification issues. In addition to accuracy, the Area Under the Curve (AUC) statistic was incorporated to evaluate the model's ability to differentiate between classes.

These configurations ensured that the model could learn well while providing useful evaluation metrics during training.

4.3 Model Training

The baseline ANN model was trained using the prepared training and validation datasets. The training process was run over a maximum of 120 epochs with a batch size of 16.

To minimize overfitting and needless training, an EarlyStopping callback was implemented. This technique checks the validation loss during training and ends the procedure if no progress is noticed for a specified number of epochs. Additionally, the best model weights were restored to ensure optimal performance.

As noted in the training logs, the model gradually increased in both accuracy and AUC during training, showing good learning of patterns in the data.

```
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8785 - auc: 0.9402 - loss: 0.3024 - val_accuracy: 0.9130 - val_auc: 0.9577 - val_loss: 0.2732
Epoch 27/120
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8840 - auc: 0.9499 - loss: 0.2837 - val_accuracy: 0.9130 - val_auc: 0.9577 - val_loss: 0.2752
Epoch 28/120
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8508 - auc: 0.9460 - loss: 0.2985 - val_accuracy: 0.9130 - val_auc: 0.9558 - val_loss: 0.2755
Epoch 29/120
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8619 - auc: 0.9393 - loss: 0.3098 - val_accuracy: 0.8913 - val_auc: 0.9538 - val_loss: 0.2785
Epoch 30/120
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8619 - auc: 0.9481 - loss: 0.2928 - val_accuracy: 0.8913 - val_auc: 0.9538 - val_loss: 0.2784
Epoch 31/120
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.8674 - auc: 0.9514 - loss: 0.2809 - val_accuracy: 0.9130 - val_auc: 0.9567 - val_loss: 0.2768
Epoch 32/120
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.9061 - auc: 0.9540 - loss: 0.2685 - val_accuracy: 0.9130 - val_auc: 0.9587 - val_loss: 0.2772
Epoch 33/120
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8619 - auc: 0.9499 - loss: 0.2756 - val_accuracy: 0.8913 - val_auc: 0.9587 - val_loss: 0.2790
Epoch 34/120
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8840 - auc: 0.9552 - loss: 0.2691 - val_accuracy: 0.8913 - val_auc: 0.9577 - val_loss: 0.2816
Epoch 35/120
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.9116 - auc: 0.9630 - loss: 0.2442 - val_accuracy: 0.8913 - val_auc: 0.9577 - val_loss: 0.2814
Epoch 36/120
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8674 - auc: 0.9518 - loss: 0.2762 - val_accuracy: 0.8913 - val_auc: 0.9587 - val_loss: 0.2814
Epoch 37/120
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8840 - auc: 0.9540 - loss: 0.2637 - val_accuracy: 0.8913 - val_auc: 0.9587 - val_loss: 0.2826
Epoch 38/120
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.8895 - auc: 0.9667 - loss: 0.2553 - val_accuracy: 0.8696 - val_auc: 0.9587 - val_loss: 0.2839
Epoch 39/120
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.9006 - auc: 0.9589 - loss: 0.2617 - val_accuracy: 0.8696 - val_auc: 0.9587 - val_loss: 0.2867
Epoch 39: early stopping
Restoring model weights from the end of the best epoch: 24.
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 57ms/step
```

Figure 5:Training log showing accuracy and loss progression during baseline ANN training.

4.4 Baseline ANN Evaluation

After training, the baseline ANN model was evaluated using the test dataset to measure its performance on unseen data. Predictions were created as probabilities and then translated into binary class labels using a threshold of 0.5.

The model attained the following performance:

- Accuracy: about 0.877
- Precision: about 0.875
- Recall: about 0.84
- F1-score: roughly 0.857
- ROC-AUC: around 0.93

These results demonstrate that the baseline ANN model performs competitively, with great classification performance as reflected by the high ROC-AUC score. The balance between precision and recall shows that the model is effective in detecting both positive and negative cases.

```
Baseline ANN Results:
              precision    recall  f1-score   support

           0       0.88      0.91      0.89        32
           1       0.88      0.84      0.86        25

    accuracy                           0.88        57
   macro avg       0.88      0.87      0.87        57
weighted avg       0.88      0.88      0.88        57

Accuracy : 0.8772
Precision: 0.875
Recall   : 0.84
F1-Score : 0.8571
ROC-AUC  : 0.93
```

Figure 6:Performance metrics of the baseline ANN model on test data.

4.5 Confusion Matrix Analysis

To further test the categorization performance, a confusion matrix was developed. The confusion matrix provides a detailed summary of right and erroneous predictions, including true positives, true negatives, false positives, and false negatives.

As seen in the confusion matrix, the model accurately categorized the majority of both positive and negative examples, with very few misclassifications. This shows that the model has a fair balance between sensitivity and specificity.

The confusion matrix also identifies regions where the model may still make errors, providing insights for additional improvement during the tuning process.

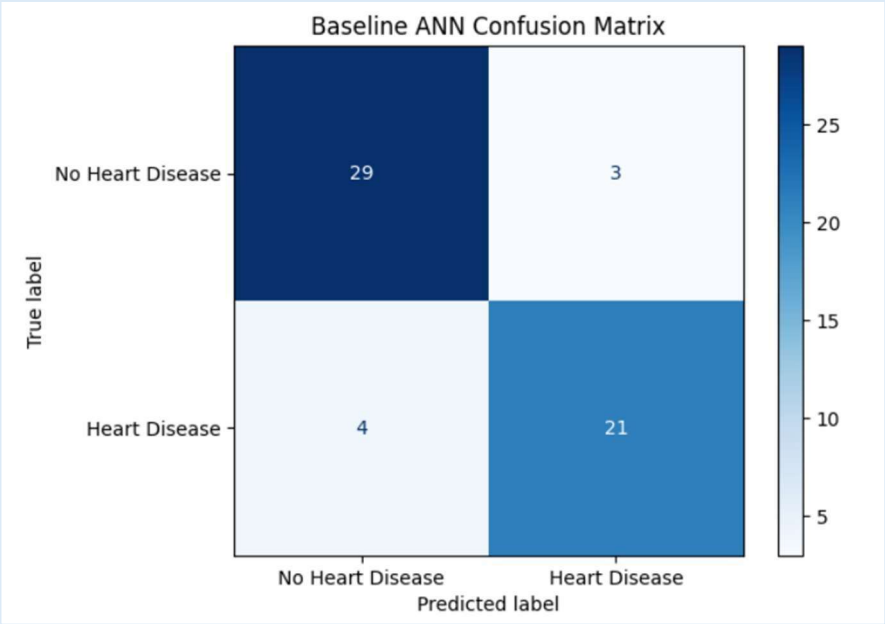


Figure 7:Confusion matrix of the baseline ANN model.

5. Hyperparametric Tuning for ANN

5.1 Tuning Strategy

After analyzing the baseline ANN model, hyperparameter tuning was undertaken to increase model performance and promote generalization. Neural networks are highly sensitive to hyperparameters such as the number of hidden layers, number of neurons, dropout rate, learning rate, and regularization strength. Improper selection of these factors might lead to overfitting or underfitting.

To overcome this, a structured two-stage tuning strategy was adopted. Instead of randomly selecting configurations, the tuning process was divided into two rounds:

- Round 1 focused on identifying the appropriate model architecture
- Round 2 focused on fine-tuning parameters around the best architecture

This approach ensured a systematic exploration of the parameter space while maintaining computing efficiency.

5.2 ANN Tuning Round 1 - Architecture Selection

In the initial phase of tuning, several ANN configurations were examined to discover the most suited architecture. The parameters studied included:

- Number of hidden layers (1, 2, and 3)
- Number of neurons in each layer
- Dropout rate
- Learning rate
- L2 regularization strength

As demonstrated in the implementation, numerous combinations were examined using the validation dataset. Each model was trained utilizing early halting and learning rate reduction approaches to achieve steady convergence.

To pick the best model, a ranking-based strategy was applied. Instead, rather than relying on a single metric, five evaluation metrics were considered:

- Validation accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

Each model was ranked across all criteria, and an average rank was derived to find the overall bestperforming configuration.

The results revealed that a model with:

- 1 hidden layer
- 32 neurons
- Dropout rate of 0.20
- Learning rate of 0.001
- L2 regularization of 0.0001was achieved the greatest overall performance in Round 1.

```
ann_round1_grid = [
    # 1 hidden layer models
    {'hidden_layers': 1, 'neurons': 16, 'dropout_rate': 0.10, 'learning_rate': 0.001,  'l2_rate': 1e-4},
    {'hidden_layers': 1, 'neurons': 32, 'dropout_rate': 0.20, 'learning_rate': 0.001,  'l2_rate': 1e-4},
    {'hidden_layers': 1, 'neurons': 48, 'dropout_rate': 0.20, 'learning_rate': 0.001,  'l2_rate': 1e-4},
    {'hidden_layers': 1, 'neurons': 64, 'dropout_rate': 0.25, 'learning_rate': 0.001,  'l2_rate': 1e-4},

    # 2 hidden layer models
    {'hidden_layers': 2, 'neurons': 32, 'dropout_rate': 0.20, 'learning_rate': 0.001,  'l2_rate': 1e-4},
    {'hidden_layers': 2, 'neurons': 48, 'dropout_rate': 0.25, 'learning_rate': 0.001,  'l2_rate': 1e-4},
    {'hidden_layers': 2, 'neurons': 64, 'dropout_rate': 0.25, 'learning_rate': 0.001,  'l2_rate': 1e-4},
    {'hidden_layers': 2, 'neurons': 64, 'dropout_rate': 0.30, 'learning_rate': 0.0007, 'l2_rate': 1e-4},

    # 3 hidden layer models
    {'hidden_layers': 3, 'neurons': 48, 'dropout_rate': 0.30, 'learning_rate': 0.0007, 'l2_rate': 1e-4},
    {'hidden_layers': 3, 'neurons': 64, 'dropout_rate': 0.30, 'learning_rate': 0.0007, 'l2_rate': 1e-4}
]
```

Figure 8:Hyperparameter search space defined for ANN tuning across different architectures.

	Hidden Layers	Neurons	Dropout	Learning Rate	L2 Rate	Validation Accuracy	Validation Precision	Validation Recall	Validation F1-Score	Validation ROC-AUC
0	1	16	0.10	0.0010	0.0001	0.804348	0.761905	0.80	0.780488	0.917308
1	1	32	0.20	0.0010	0.0001	0.891304	0.826087	0.95	0.883721	0.944231
2	1	48	0.20	0.0010	0.0001	0.847826	0.809524	0.85	0.829268	0.926923
3	1	64	0.25	0.0010	0.0001	0.847826	0.809524	0.85	0.829268	0.913462
4	2	32	0.20	0.0010	0.0001	0.847826	0.809524	0.85	0.829268	0.925000
5	2	48	0.25	0.0010	0.0001	0.847826	0.842105	0.80	0.820513	0.905769
6	2	64	0.25	0.0010	0.0001	0.847826	0.809524	0.85	0.829268	0.915385
7	2	64	0.30	0.0007	0.0001	0.869565	0.791667	0.95	0.863636	0.930769
8	3	48	0.30	0.0007	0.0001	0.891304	0.894737	0.85	0.871795	0.938462
9	3	64	0.30	0.0007	0.0001	0.869565	0.888889	0.80	0.842105	0.934615

Figure 9:Results of ANN hyperparameter tuning (Round 1) showing validation performance across configurations.

	Hidden Layers	Neurons	Dropout	Learning Rate	L2 Rate	Validation Accuracy	Validation Precision	Validation Recall	Validation F1-Score	Validation ROC-AUC	Average Rank
1	1	32	0.20	0.0010	0.0001	0.891304	0.826087	0.95	0.883721	0.944231	1.8
8	3	48	0.30	0.0007	0.0001	0.891304	0.894737	0.85	0.871795	0.938462	2.3
7	2	64	0.30	0.0007	0.0001	0.869565	0.791667	0.95	0.863636	0.930769	4.2
9	3	64	0.30	0.0007	0.0001	0.869565	0.888889	0.80	0.842105	0.934615	4.3
2	1	48	0.20	0.0010	0.0001	0.847826	0.809524	0.85	0.829268	0.926923	6.0
4	2	32	0.20	0.0010	0.0001	0.847826	0.809524	0.85	0.829268	0.925000	6.2
6	2	64	0.25	0.0010	0.0001	0.847826	0.809524	0.85	0.829268	0.915385	6.6
3	1	64	0.25	0.0010	0.0001	0.847826	0.809524	0.85	0.829268	0.913462	6.8
5	2	48	0.25	0.0010	0.0001	0.847826	0.842105	0.80	0.820513	0.905769	7.6
0	1	16	0.10	0.0010	0.0001	0.804348	0.761905	0.80	0.780488	0.917308	9.2

Best ANN Architecture from Round 1:

	Hidden Layers	Neurons	Dropout	Learning Rate	L2 Rate	Validation Accuracy	Validation Precision	Validation Recall	Validation F1-Score	Validation ROC-AUC	Average Rank
1	1	32	0.2	0.001	0.0001	0.891304	0.826087	0.95	0.883721	0.944231	1.8

Figure 10:Ranked ANN tuning results with the best-performing model identified based on average evaluation metrics.

5.3 ANN Tuning Round 2 - Fine Tuning

In the second round, the search area was restricted to focus on variants around the best architecture identified in Round 1. This allowed for more exact optimization of the model.

The tuning in this stage focused mostly on:

- Slight differences in neuron count
- Adjustments to dropout rate
- Changes in learning rate
- Regularization strength

As indicated in the implementation, many configurations were tested using an improved parameter grid. Similar to Round 1, models were evaluated using the validation dataset and graded based on numerous performance measures.

This fine-tuning procedure enables the identification of a more optimal setup while minimizing superfluous complexity.

```
ann_round2_grid = [
    {'hidden_layers': 1, 'neurons': 24, 'dropout_rate': 0.15, 'learning_rate': 0.0010, 'l2_rate': 1e-4},
    {'hidden_layers': 1, 'neurons': 32, 'dropout_rate': 0.15, 'learning_rate': 0.0010, 'l2_rate': 1e-4},
    {'hidden_layers': 1, 'neurons': 32, 'dropout_rate': 0.20, 'learning_rate': 0.0010, 'l2_rate': 1e-4},
    {'hidden_layers': 1, 'neurons': 32, 'dropout_rate': 0.25, 'learning_rate': 0.0010, 'l2_rate': 1e-4},
    {'hidden_layers': 1, 'neurons': 40, 'dropout_rate': 0.20, 'learning_rate': 0.0010, 'l2_rate': 1e-4},
    {'hidden_layers': 1, 'neurons': 48, 'dropout_rate': 0.20, 'learning_rate': 0.0010, 'l2_rate': 1e-4},
    {'hidden_layers': 1, 'neurons': 32, 'dropout_rate': 0.20, 'learning_rate': 0.0007, 'l2_rate': 1e-4},
    {'hidden_layers': 1, 'neurons': 32, 'dropout_rate': 0.20, 'learning_rate': 0.0010, 'l2_rate': 5e-4}
]
```

Figure 11:Refined hyperparameter grid for ANN tuning (Round 2) focusing on optimal parameter ranges.

	Hidden Layers	Neurons	Dropout	Learning Rate	L2 Rate	Validation Accuracy	Validation Precision	Validation Recall	Validation F1-Score	Validation ROC-AUC
0	1	24	0.15	0.0010	0.0001	0.826087	0.772727	0.85	0.809524	0.928846
1	1	32	0.15	0.0010	0.0001	0.826087	0.750000	0.90	0.818182	0.938462
2	1	32	0.20	0.0010	0.0001	0.826087	0.800000	0.80	0.800000	0.923077
3	1	32	0.25	0.0010	0.0001	0.891304	0.857143	0.90	0.878049	0.948077
4	1	40	0.20	0.0010	0.0001	0.826087	0.800000	0.80	0.800000	0.942308
5	1	48	0.20	0.0010	0.0001	0.847826	0.809524	0.85	0.829268	0.926923
6	1	32	0.20	0.0007	0.0001	0.869565	0.850000	0.85	0.850000	0.955769
7	1	32	0.20	0.0010	0.0005	0.847826	0.782609	0.90	0.837209	0.940385

Figure 12:Results of ANN hyperparameter tuning (Round 2) showing improved validation performance across configurations.

5.4 Selecting the Final Best Tuned ANN Model

Based on the outcomes of the second tuning phase, the final ANN model was picked using the same ranking approach. The best-performing configuration was:

- 1 hidden layer
- 32 neurons
- Dropout rate of 0.25
- Learning rate of 0.001
- L2 regularization of 0.0001

This setup achieved the greatest overall ranking across all evaluation measures on the validation dataset.

The rise in dropout rate relative to the baseline model suggests a larger emphasis on regularization, helping to reduce overfitting and improve generalization.

	Hidden Layers	Neurons	Dropout	Learning Rate	L2 Rate	Validation Accuracy	Validation Precision	Validation Recall	Validation F1-Score	Validation ROC-AUC	Average Rank
3	1	32	0.25	0.0010	0.0001	0.891304	0.857143	0.90	0.878049	0.948077	1.4
6	1	32	0.20	0.0007	0.0001	0.869565	0.850000	0.85	0.850000	0.955769	2.4
7	1	32	0.20	0.0010	0.0005	0.847826	0.782609	0.90	0.837209	0.940385	3.7
5	1	48	0.20	0.0010	0.0001	0.847826	0.809524	0.85	0.829268	0.926923	4.5
1	1	32	0.15	0.0010	0.0001	0.826087	0.750000	0.90	0.818182	0.938462	5.3
4	1	40	0.20	0.0010	0.0001	0.826087	0.800000	0.80	0.800000	0.942308	5.8
0	1	24	0.15	0.0010	0.0001	0.826087	0.772727	0.85	0.809524	0.928846	6.1
2	1	32	0.20	0.0010	0.0001	0.826087	0.800000	0.80	0.800000	0.923077	6.8

Final Tuned ANN Parameters:

	Hidden Layers	Neurons	Dropout	Learning Rate	L2 Rate	Validation Accuracy	Validation Precision	Validation Recall	Validation F1-Score	Validation ROC-AUC	Average Rank
3	1	32	0.25	0.001	0.0001	0.891304	0.857143	0.9	0.878049	0.948077	1.4

Figure 13:Final ranked ANN tuning results highlighting performance across evaluation metrics.

5.5 Discussion of Tuning Results

The hyperparameter tuning technique yielded some crucial insights:

- Simpler architectures with fewer layers fared better than deeper networks, perhaps due to the very limited dataset size
- A modest number of neurons (32) provided the optimal balance between learning capacity and generalization
- Increasing dropout enhanced model resilience by reducing overfitting
- Fine-tuning learning rate and regularization contributed to better stability

Overall, the tuning process proved that careful optimization of hyperparameters can considerably influence ANN performance. Rather than adding complexity, selecting the proper combination of parameters was crucial to attaining optimal results.

This structured tuning approach ensured that the final ANN model achieved an optimal balance between performance and generalization, while avoiding unnecessary model complexity.

6. Final Selected ANN Model Evaluation.

6.1 Performance Matrix

After hyperparameter tuning, the final ANN model was evaluated on the test dataset to measure its performance on unseen data. The evaluation was conducted using five main metrics: accuracy, precision, recall, F1-score, and ROC-AUC.

As demonstrated in the results, the tuned ANN attained the following performance:

- Accuracy: about 0.8596
- Precision: about 0.840
- Recall: about 0.84
- F1-score: roughly 0.84
- ROC-AUC: around 0.95

Although the overall accuracy is slightly lower than the baseline ANN model, the ROC-AUC score improved dramatically. This suggests that the tuned model has a higher ability to distinguish between patients with and without cardiac disease across different categorization thresholds.

The balance between precision and recall shows that the model performs consistently in detecting both positive and negative examples, making it suited for medical prediction tasks where both types of errors are relevant.

```
Final Tuned ANN Results:
              precision    recall  f1-score   support

           0       0.88      0.88      0.88        32
           1       0.84      0.84      0.84        25

    accuracy                           0.86        57
   macro avg       0.86      0.86      0.86        57
weighted avg       0.86      0.86      0.86        57

Accuracy : 0.8596
Precision: 0.84
Recall   : 0.84
F1-Score : 0.84
ROC-AUC  : 0.95
```

Figure 14:Final performance metrics of the tuned ANN model..

6.2 Confusion Matrix Analysis

To further analyse the classification performance, a confusion matrix was built for the final ANN model. The confusion matrix provides a detailed breakdown of prediction outcomes, including true positives, true negatives, false positives, and false negatives.

The findings demonstrate that the model accurately classifies a majority of both classes, with relatively few misclassifications. This shows that the model has found a reasonable balance between sensitivity (recall) and specificity.

In the context of heart disease prediction, avoiding false negatives is particularly crucial, as failing to recognize a patient with heart disease might have catastrophic repercussions. The model displays a reasonable capacity to catch positive cases while preserving overall accuracy.

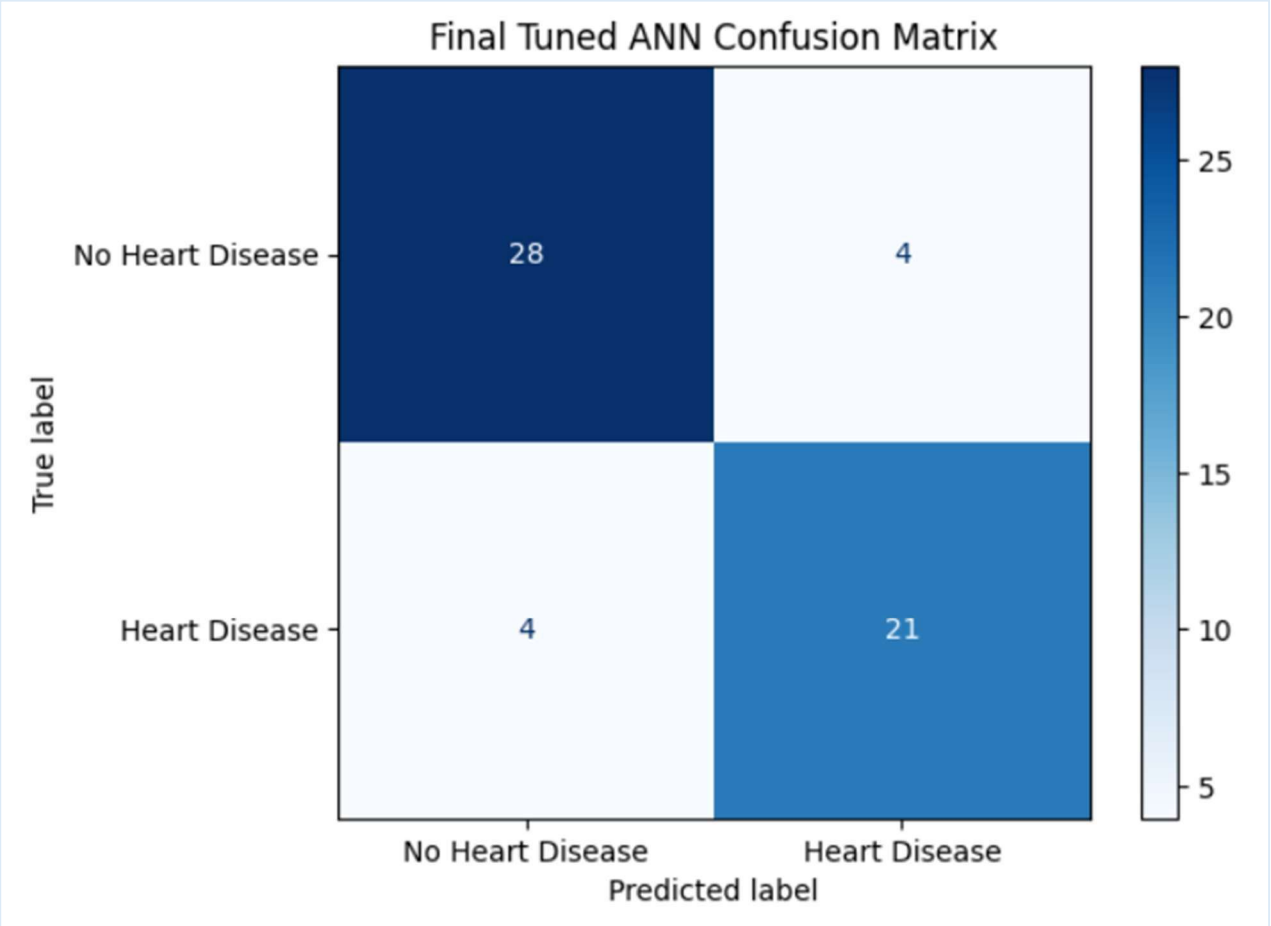


Figure 15:Confusion matrix of the final tuned ANN model.

6.3 Training and Validation Curves

To study the learning behaviour of the ANN model, training and validation curves were examined. These curves indicate how the model's performance evolved across epochs during training.

As seen in the training plots:

- The training and validation accuracy curves indicate gradual improvement and eventual stabilization
- The loss curves shrink over time, showing good learning
- There is no substantial divergence between training and validation curves

This shows that the model has trained adequately without severe overfitting. The use of dropout layers and early stopping helped to steady training and increased generalization.

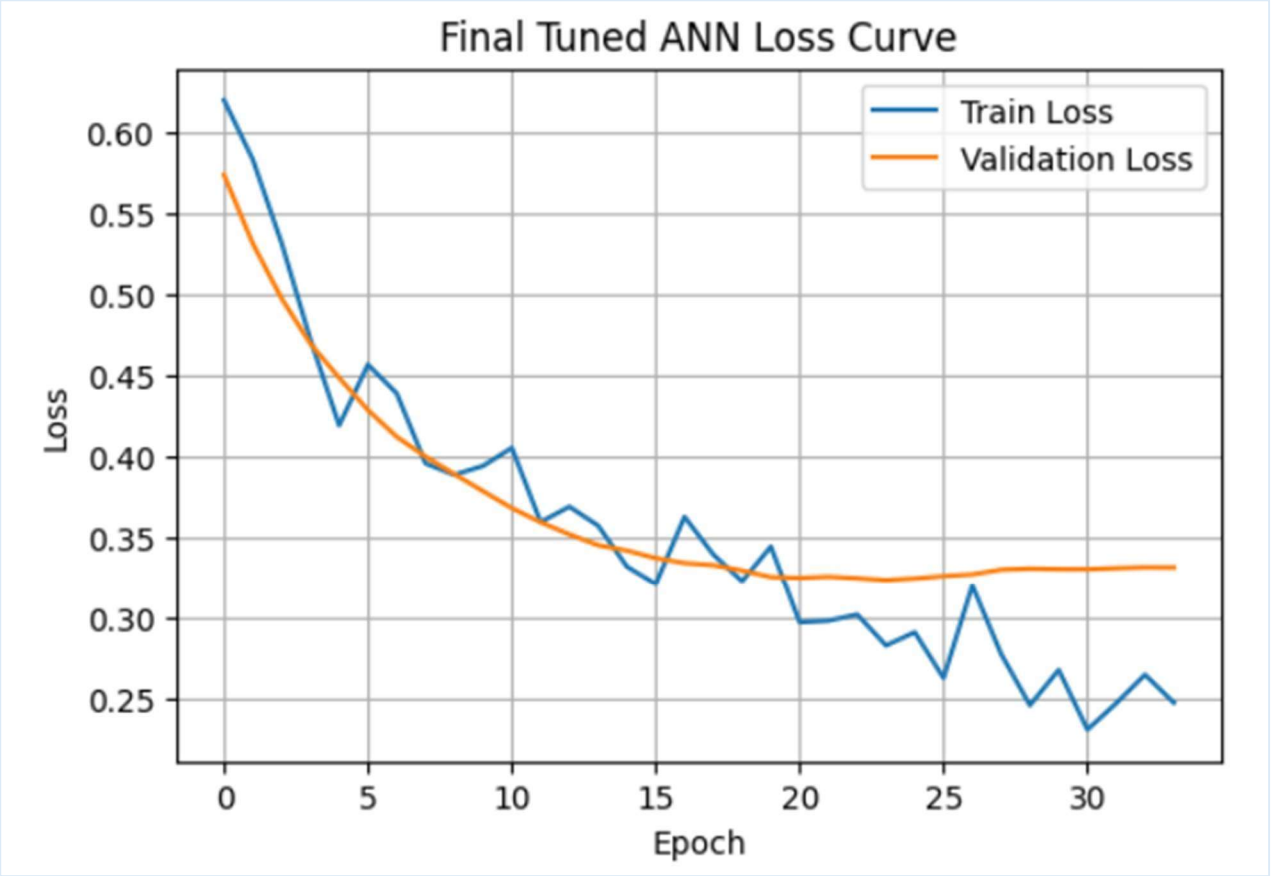


Figure 16:Training and validation loss curves of the tuned ANN model.

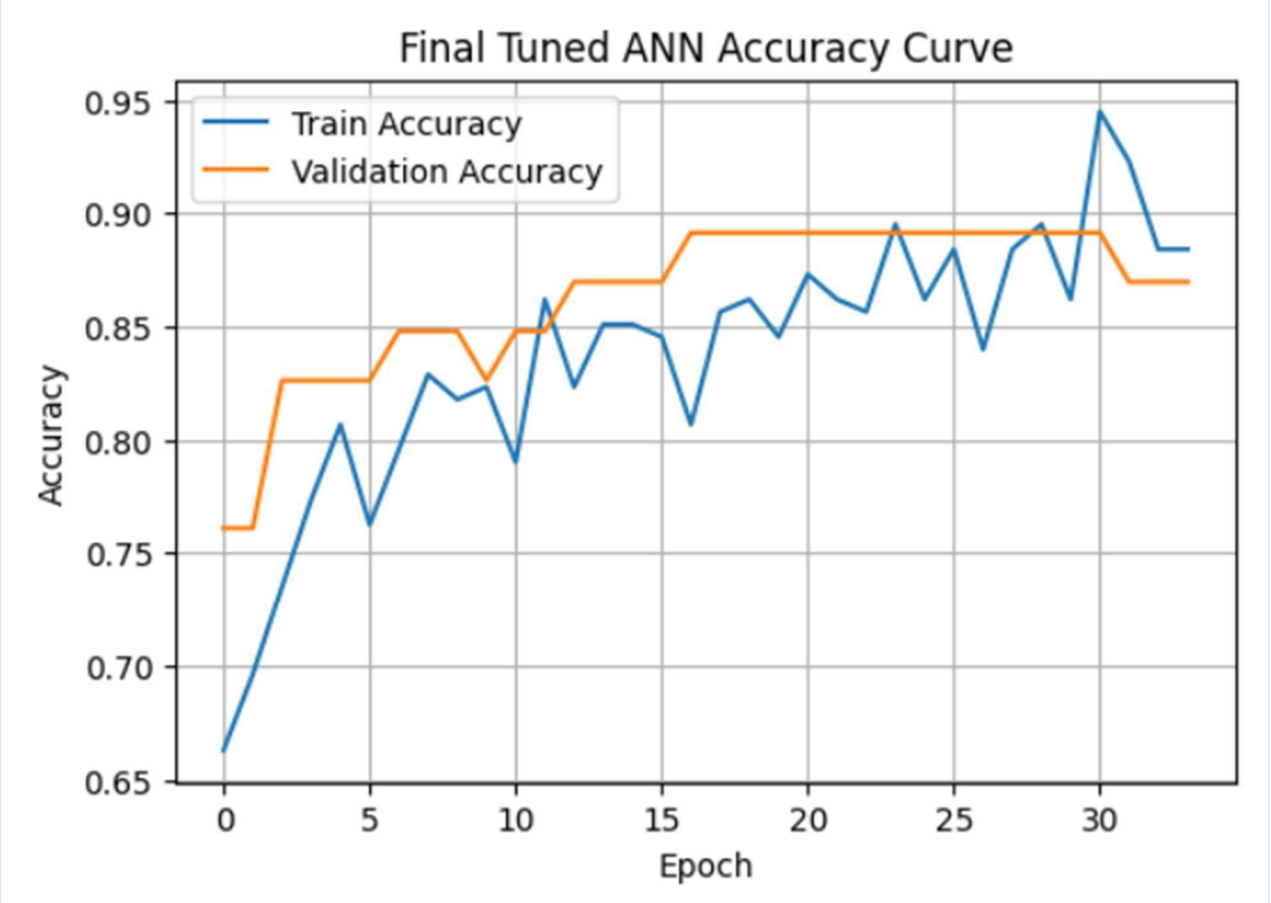


Figure 17:Training and validation accuracy curves of the tunedl ANN model.

6.4 ROC Curve Analysis

The Receiver Operating Characteristic (ROC) curve was plotted to examine the classification performance of the ANN model across multiple threshold settings.

The ROC curve for the tuned ANN model exhibits a great performance, with an Area Under the Curve (AUC) of roughly 0.95. The curve is positioned close to the top-left corner of the graph, indicating high true positive rates and low false positive rates.

This result demonstrates that the ANN model is highly effective in discriminating between the two classes, even if the classification threshold varies. In medical applications, such performance is particularly significant since it offers flexibility in altering thresholds based on risk tolerance.

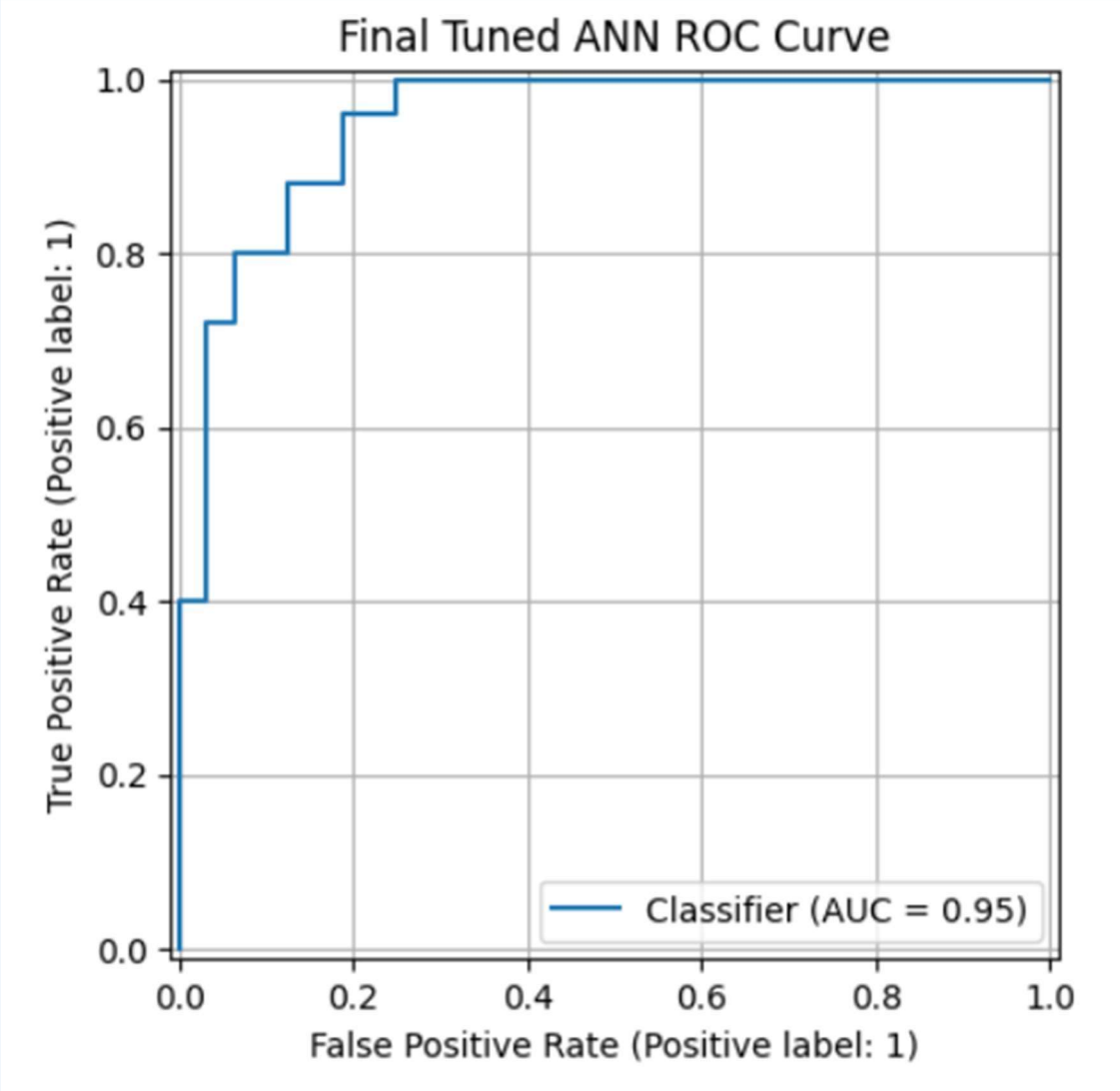


Figure 18:ROC curve of the final ANN model..

6.5 Comparison across Baseline vs Tuned ANN

A comparison between the baseline ANN and the tuned ANN was undertaken to determine the impact of hyperparameter tuning.

As shown in the comparison findings:

- The baseline ANN achieved slightly higher accuracy (~0.877)
- The Tuned ANN achieved a higher ROC-AUC (~0.95 compared to ~0.93)
- Other parameters such as precision and recall were generally constant

This suggests that while the modified model trades a tiny amount of accuracy, it considerably improves its capacity to distinguish across classes. Therefore, the modified ANN model delivers higher overall classification performance, particularly in terms of generalization.

	Accuracy	Precision	Recall	F1-Score	ROC-AUC	Average Rank
Baseline ANN	0.8772	0.875	0.84	0.8571	0.93	1.3
Tuned ANN	0.8596	0.840	0.84	0.8400	0.95	1.7

Figure 19:Comparison of baseline and tuned ANN performance across evaluation metrics.

Overall, the tuned ANN model demonstrates improved generalization and classification capability, as evidenced by its higher ROC-AUC score, despite a slight reduction in accuracy.

7. Feature Importance and Interpretability

7.1 Importance of Model Interpretability in ANN.

In predictive modeling, particularly in healthcare applications, interpretability is vital for assuring transparency, confidence, and practical applicability. While strong predictive performance is desirable, understanding how input variables influence model predictions is equally crucial for clinical validation and decision-making. Models that lack interpretability may give accurate findings but are typically difficult to justify in real-world circumstances (Rudin, 2019).

Traditional machine learning models, especially tree-based algorithms such as Random Forest and Decision Tree, intrinsically generate feature importance estimates based on impurity reduction or information gain. These measures allow direct identification of influential variables. However, Artificial Neural Networks (ANNs) do not give such clear interpretability due to their multi-layered structure and dispersed weight representation (Goodfellow, et al., 2016).

As a result, ANN models are usually referred to as “black-box” models. This constraint involves the employment of external, model-agnostic tools to quantify feature relevance and analyse model behaviour.

7.2 Computation of ANN Feature Importance using Permutation Method

To overcome the lack of intrinsic interpretability in ANN models, the permutation significance technique was adopted. Permutation significance measures the contribution of each feature by assessing the drop in model performance when the values of that feature are randomly shuffled. If a feature is crucial, permuting it will break the link with the target variable, resulting to a notable drop

in model accuracy. Conversely, traits with limited importance will have negligible impact on performance when permuted (Breiman, 2001; Molnar, 2020).

In this study, a custom ANN wrapper class was constructed to ensure compatibility with the Scikitlearn permutation significance function. The evaluation was conducted on the test dataset with the following configuration:

- Number of repetitions: 20
- Evaluation metric: accuracy

The computed permutation importance ratings revealed that:

- ca_0 (0.0386) and ca_2 (0.0333) are the most influential features
- thalach (0.0175) and restecg variables (~0.0123) also indicate considerable importance
- Several traits have near-zero importance
- Some features have negative relevance values

The existence of negative significance values shows that permuting specific variables slightly improved model performance, suggesting that certain features may generate noise or redundancy (Molnar, 2020).

```
ANN Permutation Importance:
ca_0         3.859649e-02
ca_2         3.333333e-02
thalach      1.754386e-02
restecg_0    1.228070e-02
restecg_2    1.228070e-02
oldpeak      6.140351e-03
chol         6.140351e-03
cp_3         4.385965e-03
slope_0      4.385965e-03
ca_1         8.771930e-04
fbs_0        1.110223e-17
cp_0         0.000000e+00
restecg_1    0.000000e+00
ca_3        -8.771930e-04
slope_2     -8.771930e-04
thal_3      -2.631579e-03
fbs_1       -2.631579e-03
sex_1       -3.508772e-03
thal_2      -3.508772e-03
exang_0     -7.017544e-03
exang_1     -7.017544e-03
trestbps    -8.771930e-03
sex_0       -1.052632e-02
ï»¿age      -1.140351e-02
thal_1      -1.140351e-02
cp_2        -1.403509e-02
cp_1        -1.929825e-02
slope_1     -2.105263e-02
dtype: float64
```

Figure 20:Permutation importance scores for all features in the ANN model.

7.3 Interpretation of ANN Feature Importance Results

The permutation importance results indicate that the ANN model depends mostly on clinically important features. In particular:

- The number of significant vessels (ca_0, ca_2) has the highest relevance, indicating its excellent predictive power
- Maximum heart rate (thalach) plays a crucial impact in classification
- Electrocardiographic characteristics (restecg) contribute considerably to predictions
- ST depression (oldpeak) and cholesterol both exhibit moderate importance
- These findings are congruent with current medical knowledge, where such factors are known indications of cardiac disease.

Additionally, features such as cp_1, slope_1, and age have negative significance values, suggesting that they may not contribute favorably to the ANN model. This demonstrates the ability of permutation significance to discover both essential and potentially irrelevant features.

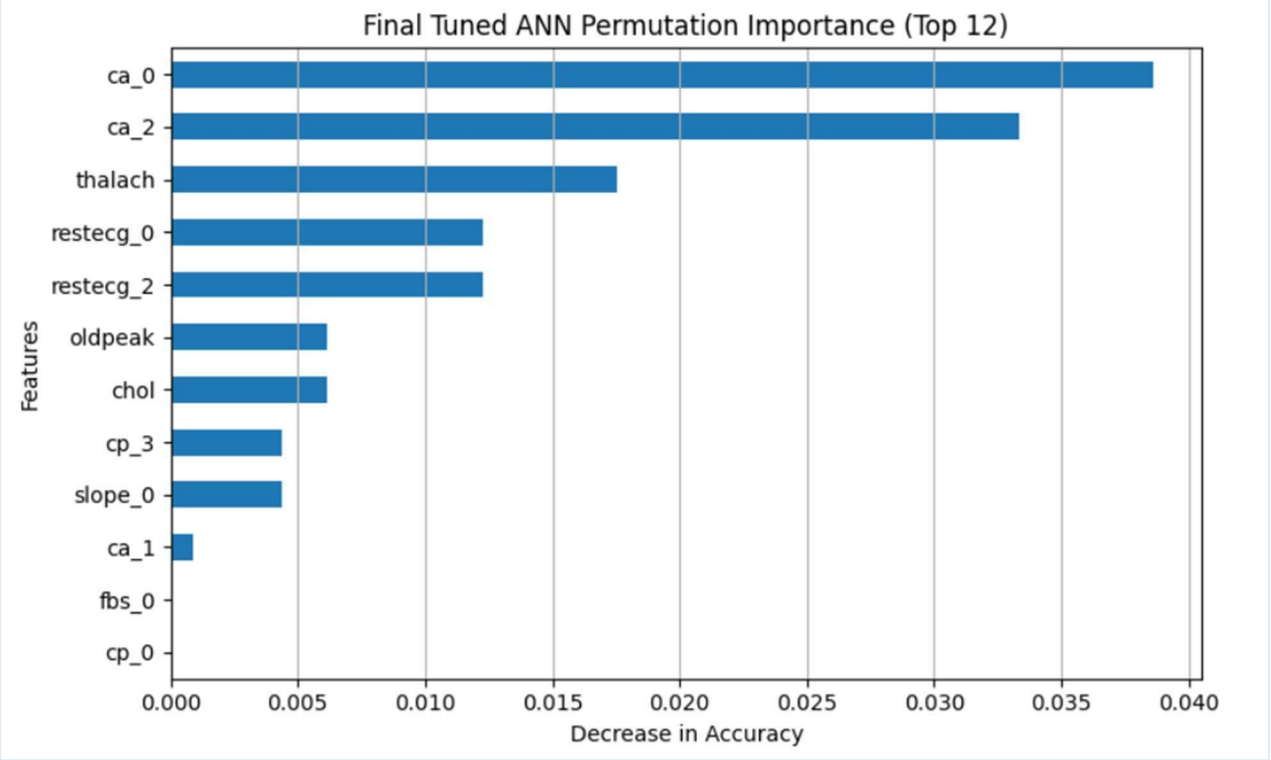


Figure 21:Top 12 important features based on ANN permutation importance.

7.4 Feature Importance in Tree-Based Models

To provide a comparative viewpoint, feature importance scores were collected from tree-based models, including Random Forest, Decision Tree, AdaBoost, and XGBoost.

These models calculate feature relevance depending on how frequently and successfully features are utilized to partition the data. Unlike permutation importance, which assesses influence on performance, tree-based important highlights the structural role of characteristics in decisionmaking.

The significance values from various models were integrated with ANN permutation importance into a uniform DataFrame, allowing direct comparison across models.

	Random Forest	Decision Tree	AdaBoost	XGBoost	ANN (Permutation)
ca_0	0.082355	0.139753	0.165618	0.069427	0.038596
ca_1	0.018334	0.000000	0.000000	0.042166	0.000877
ca_2	0.007482	0.000000	0.000000	0.000000	0.033333
ca_3	0.004715	0.000000	0.000000	0.000000	-0.000877
chol	0.029531	0.000000	0.016289	0.031278	0.006140

Figure 22:Feature importance comparison across tree-based models and ANN (sample features).

7.5 Aggregation and Selection of Top Features.

To enable a fair comparison between models, the average relevance score for each feature was computed. Based on this aggregated assessment, the top 12 attributes were picked.

This technique avoids model-specific bias and highlights features that are consistently important across different algorithms.

The selected top features include:

- cp_3
- thal_3
- ca_0
- oldpeak
- thalach

This indicates that some features continuously contribute to prediction regardless of the modeling approach.

	Random Forest	Decision Tree	AdaBoost	XGBoost	ANN (Permutation)
cp_3	0.133936	0.435198	0.240955	0.174380	0.004386
thal_3	0.123543	0.238288	0.152461	0.107600	-0.002632
ca_0	0.082355	0.139753	0.165618	0.069427	0.038596
oldpeak	0.068739	0.071269	0.187902	0.060553	0.006140
thal_1	0.129639	0.000000	0.076295	0.185330	-0.011404
thalach	0.084744	0.069482	0.096308	0.037369	0.017544
exang_1	0.068446	0.000000	0.027660	0.034181	-0.007018
exang_0	0.061219	0.000000	0.019397	0.042993	-0.007018
ï»¿age	0.030354	0.046009	0.000000	0.037744	-0.011404
chol	0.029531	0.000000	0.016289	0.031278	0.006140
slope_0	0.018996	0.000000	0.000000	0.038612	0.004386
ca_1	0.018334	0.000000	0.000000	0.042166	0.000877

Figure 23:Top features selected based on aggregated importance across models.

7.6 Comparison of Feature Importance Across Models

The feature importance comparison plot provides a visual picture of how different models allocate importance to features.

Key observations from the figure include:

- Strong agreement among models for features such as ca_0, ca_2, thalach, and oldpeak
- Tree-based models award higher priority values due to their split-based calculating technique
- ANN significance values are significantly less, as they indicate performance impact rather than structural contribution
- Some variation exists between models, showing differences in how patterns are learned

Despite these changes, the consistency in important features among models raises confidence in the robustness of the results.

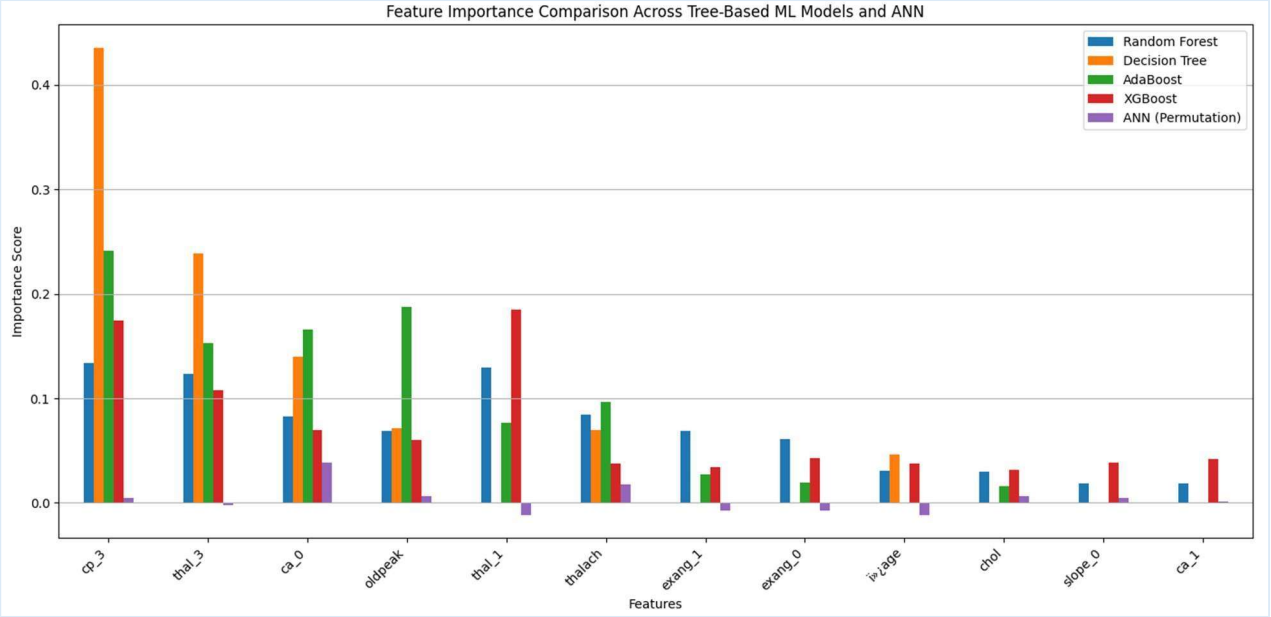


Figure 24:Comparison of feature importance across tree-based models and ANN.

7.7 Justification for Using Permutation Importance

Permutation Importance was selected due to its applicability for ANN models and its interpretability advantages.

Firstly, it is a model-agnostic method, meaning it may be applied to any model, including neural networks. Secondly, it explicitly assesses the influence of each feature on model performance, making it intuitive and matched with evaluation metrics such as accuracy.

Alternative approaches such as SHAP and LIME were considered. However, SHAP is computationally expensive, particularly for neural networks, while LIME provides local explanations rather than a global view of feature importance (Lundberg & Lee, 2017).

Therefore, permutation importance provides an effective compromise between interpretability and computational efficiency.

7.8 Key Insights from Feature Performance

The feature importance study indicates that both ANN and classic machine learning models consistently identify a core set of relevant characteristics. This consistency among models increases the dependability of the conclusions and demonstrates that the models are capturing relevant patterns in the data.

Furthermore, the ANN model, despite its black-box nature, may be effectively analysed utilizing permutation significance. This facilitates the extraction of actionable insights while maintaining strong prediction performance.

These results underscore the relevance of combining predictive modeling with interpretability techniques, particularly in healthcare applications where comprehending model decisions is critical.

8. Overall Model Comparison across All Tuned ML Models and Tuned ANN Model

8.1 Consolidated Performance Evaluation

To give a full evaluation of all models, the performance characteristics of all tuned machine learning models and the final tuned ANN were consolidated into a single comparison table .

The review was conducted utilizing five major metrics:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC

This multi-metric method ensures that model performance is examined holistically rather than depending on a single indication.

From the results:

- Logistic Regression achieved the greatest accuracy (~ 0.8947)
- Support Vector Machine demonstrated consistently good performance across all measures
- ANN achieved somewhat lesser accuracy (~ 0.8596) but the best ROC-AUC (~ 0.95)
- Tree-based models (Random Forest, AdaBoost, XGBoost) demonstrated moderate and steady performance
- Decision Tree performed the weakest, showing overfitting tendencies.

	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.8947	0.9130	0.84	0.8750	0.9325
Random Forest	0.8421	0.8636	0.76	0.8085	0.9138
SVM	0.8772	0.8750	0.84	0.8571	0.9275
Decision Tree	0.7368	0.7083	0.68	0.6939	0.7950
AdaBoost	0.7895	0.7826	0.72	0.7500	0.9081
XGBoost	0.7895	0.7826	0.72	0.7500	0.8888
ANN	0.8596	0.8400	0.84	0.8400	0.9500

Figure 25:Performance comparison of all tuned ML models and the final ANN across evaluation metrics.

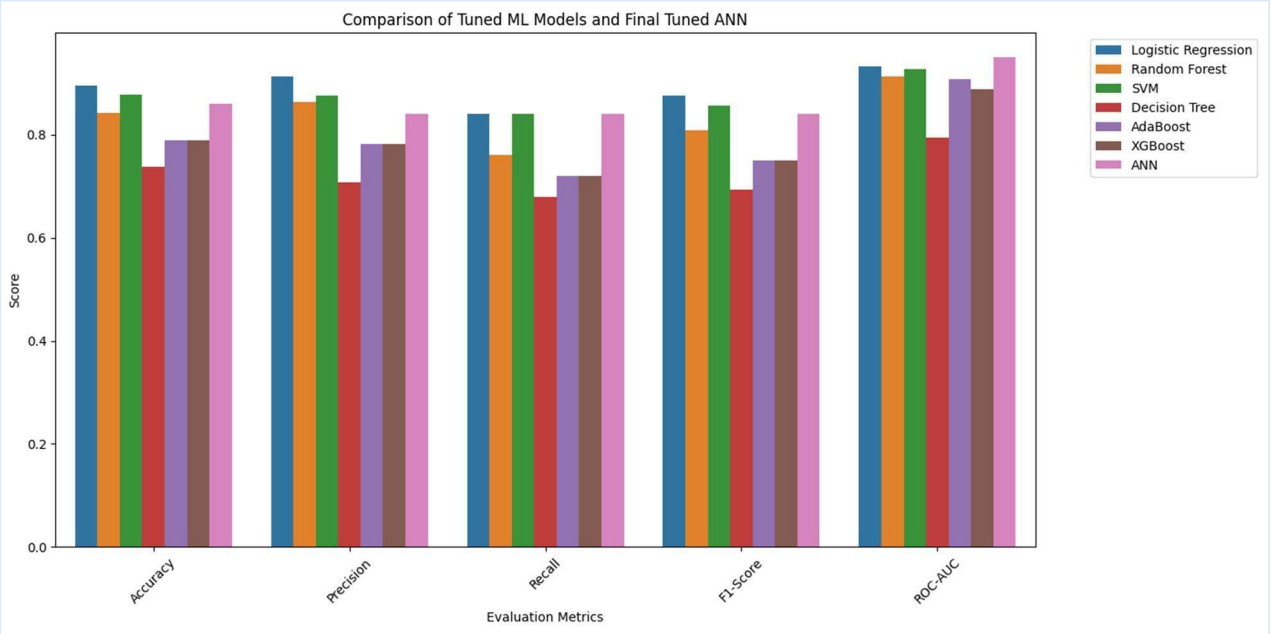


Figure 26:Comparison of model performance across accuracy, precision, recall, F1-score, and ROC-AUC.

8.2 Metric-wise Comparative Analysis

A detailed investigation of individual measures indicates substantial variances in model behavior:

- Accuracy: Logistic Regression achieved the maximum accuracy, showing that the dataset is somewhat organized and linearly separable. This explains why simpler models outperform more complex ones.
- Precision and Recall: Most models demonstrate roughly balanced precision and recall, indicating that they are effective in identifying both positive and negative cases. ANN also maintains balanced values, suggesting consistent categorization behavior.
- F1-Score: The F1-score trends closely track precision and recall. Logistic Regression and SVM yield the greatest values, while ANN stays competitive.
- ROC-AUC: The ANN model scored the best ROC-AUC (~ 0.95), showing superior ability to differentiate between classes across different thresholds. This illustrates the ANN's strength in probabilistic classification rather than rigid classification accuracy.

8.3 ROC Curve Comparison

The ROC curve comparison provides a visual depiction of model performance across thresholds.

Key observations:

- ANN curve is closest to the top-left corner which indicates highest discrimination ability
- Logistic Regression and SVM both display significant curves
- Tree-based models demonstrate moderate performance
- Decision Tree curve is weakest

This demonstrates that while ANN may not have the highest accuracy, it performs best in discriminating across classes, which is critical in medical prediction tasks.

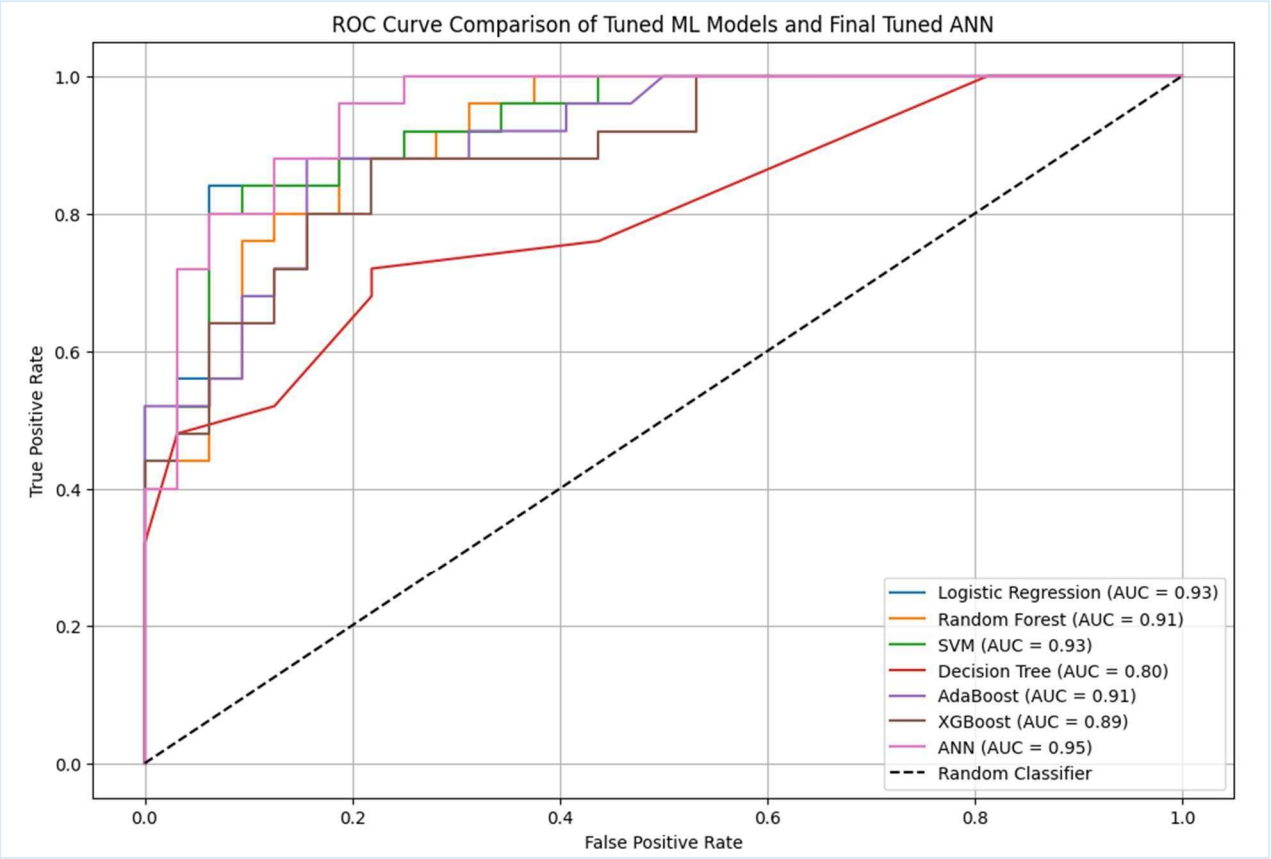


Figure 27:ROC curve comparison of all tuned ML models and the ANN model.

8.4 Model Ranking and Best Model Selection

To ensure a fair comparison, a ranking system was established using all five evaluation matrices.

Each model was ranked for each statistic, and an average rank was calculated.

The final ranking results indicate:

- Logistic Regression
- Support Vector Machine
- Artificial Neural Network (ANN)
- Random Forest
- AdaBoost
- XGBoost
- Decision Tree

The best overall model was found as Logistic Regression, based on its consistently excellent performance across all metrics.

	Accuracy	Precision	Recall	F1-Score	ROC-AUC	Average Rank
Logistic Regression	0.8947	0.9130	0.84	0.8750	0.9325	1.4
SVM	0.8772	0.8750	0.84	0.8571	0.9275	2.2
ANN	0.8596	0.8400	0.84	0.8400	0.9500	2.6
Random Forest	0.8421	0.8636	0.76	0.8085	0.9138	3.8
AdaBoost	0.7895	0.7826	0.72	0.7500	0.9081	5.4
XGBoost	0.7895	0.7826	0.72	0.7500	0.8888	5.6
Decision Tree	0.7368	0.7083	0.68	0.6939	0.7950	7.0

```
Best Overall Model Considering All Five Metrics: Logistic Regression

Best Model Performance:
```

	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.8947	0.913	0.84	0.875	0.9325

Figure 28:Final ranking of models based on average performance across all evaluation metrics.

8.5 Key Insights from Model Comparison

The overall comparison of modified machine learning models and the final ANN model shows various key insights regarding model behavior, dataset properties, and prediction performance.

Firstly, simpler models such as Logistic Regression achieved the highest accuracy among all models. This shows that the dataset exhibits a relatively structured nature with a degree of linear separability. As explained by (James, et al., 2021), linear models perform effectively when

interactions between variables are approximately linear, as they may capture patterns without introducing excessive complexity.

Secondly, the ANN model displayed improved performance in terms of ROC-AUC. According to (Goodfellow, et al., 2016), neural networks are particularly effective at capturing nonlinear relationships and complicated feature interactions. This explains why the ANN obtains higher discrimination capability, even if improvements in accuracy are rather small.

Thirdly, ensemble models such as Random Forest, AdaBoost, and XGBoost exhibited robust and consistent performance across all assessment measures. (Breiman, 2001) indicates that ensemble approaches improve prediction accuracy by lowering variation and merging numerous learners, which contributes to their robustness.

Finally, the Decision Tree model displayed the worst performance, perhaps due to overfitting. As stated by (Hastie, et al., 2009), decision trees are prone to overfitting because they might build extremely complicated structures that fit the training data closely but fail to generalize well.

Overall, these findings illustrate that model performance is highly reliant on dataset characteristics, and that simpler models can outperform more complex approaches when the data does not require extensive nonlinear modelling.

8.6 Trade-offs Between Models

The comparison between ANN and classic machine learning models reveals some critical trade-offs that must be addressed while picking a suitable model.

One significant trade-off is between accuracy and generality. Logistic Regression achieved the maximum accuracy, making it suited for direct classification jobs. However, the ANN model achieved a higher ROC-AUC score, indicating improved generalization across multiple thresholds. (Goodfellow, et al., 2016) Demonstrate that neural networks are capable of learning complicated decision boundaries, which boosts their ability to generalize beyond simple categorization rules.

Another big trade-off emerges between interpretability and model complexity. Traditional models such as Logistic Regression and Decision Trees are naturally interpretable, allowing direct

knowledge of feature contributions. In contrast, ANN models require extra interpretability strategies such as permutation importance. (Rudin, 2019) Notes that more complicated models often forfeit interpretability, which might be a constraint in key fields such as healthcare.

Computational efficiency is also an essential element. Traditional machine learning methods are often faster to train and use fewer computer resources. In contrast, neural networks involve iterative optimization methods, making them more computationally intensive. (Chollet, 2017) Highlights that deep learning models require careful tuning and longer training times compared to standard techniques.

Additionally, model stability differs between approaches. Ensemble approaches give increased resilience by minimizing variance, but ANN models are more sensitive to hyperparameter selection. (Hastie, et al., 2009) Stress that ensemble techniques promote stability by merging several models, whereas single complicated models require careful optimization.

These trade-offs illustrate that no single model is generally ideal, and model selection should be led by the specific requirements of the task, including performance, interpretability, and computational restrictions.

8.7 Overall Conclusion from Comparison

The overall comparison of tuned machine learning models and the final ANN model indicates that different models excel in different aspects of performance, and no single strategy dominates across all evaluation measures.

Logistic Regression emerged as the best overall model due to its high accuracy, simplicity, and consistent performance across all measures. Its effectiveness shows that the dataset is wellstructured and does not require highly sophisticated modeling techniques.

The ANN model, on the other hand, generated the greatest ROC-AUC score, indicating improved classification power and better generalization across multiple decision thresholds. This demonstrates the strength of ANN in capturing deeper patterns within the data, even if it does not achieve the best accuracy.

Tree-based models offered a mix between performance and interpretability but did not significantly outperform simpler models. The Decision Tree model, in particular, exhibited shortcomings due to overfitting, reinforcing the necessity for ensemble techniques or regularization.

These findings suggest that the choice of model should not be based exclusively on a single performance parameter. Instead, it should examine various aspects, including predictive accuracy, generalization ability, interpretability, and computing economy.

The overall comparison demonstrates that while traditional models such as Logistic Regression achieve superior accuracy on structured datasets, Artificial Neural Networks provide enhanced classification capability and generalization, highlighting the importance of selecting models based on both performance metrics and application requirements.

9. Overall Results, Insights, Challenges and Conclusion

9.1 Summary of Results

This study intended to construct and assess predictive models for heart disease classification using both classic machine learning approaches and Artificial Neural Networks (ANN). A detailed evaluation was conducted utilizing numerous performance criteria, including accuracy, precision, recall, F1-score, and ROC-AUC.

The results suggest that among all calibrated machine learning models, Logistic Regression scored the highest overall accuracy (about 0.8947), followed closely by Support Vector Machine. These models displayed consistent and balanced performance across all evaluation metrics, showing their applicability for structured datasets.

The ANN model, despite attaining a slightly lower accuracy (about 0.8596), displayed greater performance in terms of ROC-AUC (around 0.95). This illustrates its remarkable capacity to discriminate across classes across varying decision thresholds. Ensemble models such as Random Forest, AdaBoost, and XGBoost gave consistent but middling performance, while the Decision Tree model exhibited the poorest performance due to overfitting.

Overall, the results reveal that different models excel in different aspects of performance, underlining the significance of evaluating models using multiple measures rather than relying on a single measure.

9.2 Overall Key Insights

Several noteworthy insights were identified from the investigation. Firstly, the good performance of Logistic Regression implies that the dataset has a relatively structured nature with a degree of linear separability. As explained by (James, et al., 2021), linear models perform effectively when

relationships between variables are not excessively complicated, allowing them to generate good results without requiring advanced modeling techniques.

Secondly, the ANN model displayed higher ROC-AUC performance, indicating enhanced classification capabilities and generalization. According to (Goodfellow, et al., 2016), neural networks are particularly adept at capturing nonlinear relationships and complex feature interactions, which explains their ability to achieve high discrimination performance even when accuracy differences are minor.

Thirdly, feature importance analysis demonstrated that critical clinical variables such as the number of main vessels (ca), maximal heart rate (thalach), and electrocardiographic results (restecg) significantly influenced predictions across both ANN and traditional models. This consistency increases the dependability of the findings and demonstrates that the models are capturing relevant patterns in the data.

Finally, the comparison of models indicates that increased model complexity does not necessarily ensure improved performance. Instead, the effectiveness of a model depends largely on the properties of the dataset.

9.3 Challenges and Limitations

Several problems were faced during the implementation and analysis of this study. One notable disadvantage was the relatively small dataset size. Neural networks often require significant volumes of data to fully maximize their learning capacity. As a result, the ANN model may not have attained its full potential in this investigation.

Another problem was the intricacy of hyperparameter tuning for the ANN model. Unlike standard machine learning models, neural networks involve careful adjustment of numerous parameters, including the number of layers, neurons, learning rate, and regularization approaches. This approach was computationally intensive and required numerous rounds to attain optimal performance (Chollet, 2017).

Interpretability also offered a hurdle, notably for the ANN model. While tree-based models provide direct feature importance estimates, ANN models needed the use of permutation importance to assess feature contributions. As stated by (Rudin, 2019), the lack of interpretability in complicated models can be a hindrance in high-stakes areas such as healthcare.

Additionally, some characteristics revealed negative important values in the permutation importance analysis, indicating probable noise or redundancy in the dataset. This shows that further feature selection or dimensionality reduction strategies could improve model performance.

9.4 Final Conclusion

In conclusion, this study reveals that both classic machine learning models and Artificial Neural Networks can be efficiently employed for heart disease prediction, each giving distinct advantages.

Logistic Regression emerged as the best overall model due to its high accuracy, simplicity, and consistency across all evaluation metrics. Its great performance illustrates the effectiveness of simpler models for structured datasets.

The ANN model, while somewhat less accurate, displayed improved classification power and generalization, as evidenced by its higher ROC-AUC score. This highlights its potential for capturing complicated linkages within the data.

The conclusions of this study show that model selection should not be based exclusively on a single performance parameter. Instead, it should examine various aspects, including predictive accuracy, generalization ability, interpretability, and computing economy.

The overall comparison demonstrates that while traditional models such as Logistic Regression achieve superior accuracy on structured datasets, Artificial Neural Networks provide enhanced classification capability and generalization, highlighting the importance of selecting models based on both performance metrics and application requirements.

References

Bishop, C. M., 2006. Pattern Recignition and Machine Learning. New York: Springer.

Breiman, L., 2001. Random Forests. Machine Learning, 45(1), pp. 5-32.

Chollet, F., 2017. Deep Learning with Python. New York: Manning Publications.

Goodfellow, I., Bengio, Y. & Courville, A., 2016. Deep Learning. Cambridge, MA: MIT Press.

Hastie, T., Tibshirani, R. & Friedman, J., 2009. The Elements of Statistical Learning. 2 ed. New York: Springer.

Haykin, S., 2009. Neural Networks and Learning Machines. 3 ed. Upper Saddle River, NJ: Pearson.

James, G., Witten, D., Hastie, T. & Tibshirani, R., 2021. An Introduction to Statistical Learning. 2 ed. New York: Springer.

LeCun, Y., Bengio, Y. & Hinton, G., 2015. Deep Learning. Nature, 521(7553), pp. 436-444.

Lundberg, S. M. & Lee, S.-I., 2017. A unified approach to interpreting model predictions. s.l., s.n.

Molnar, C., 2020. Interpretable Machine Learning. [Online] Available at: https://christophm.github.io/interpretable-ml-book/ [Accessed 26 April 2026].

Rudin, C., 2019. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. Nature Machine Intelligence, 1(5), pp. 206-215.